



A Practical Workflow for Fuzzing and Scanning in Bug Bounty

How to Systematically expand your attack surface, eliminate noise and find the bugs others miss.

□□□□□□□

Published March 26, 2026

Free: No

[Read on Freedium](#) · [original](#)

Contents

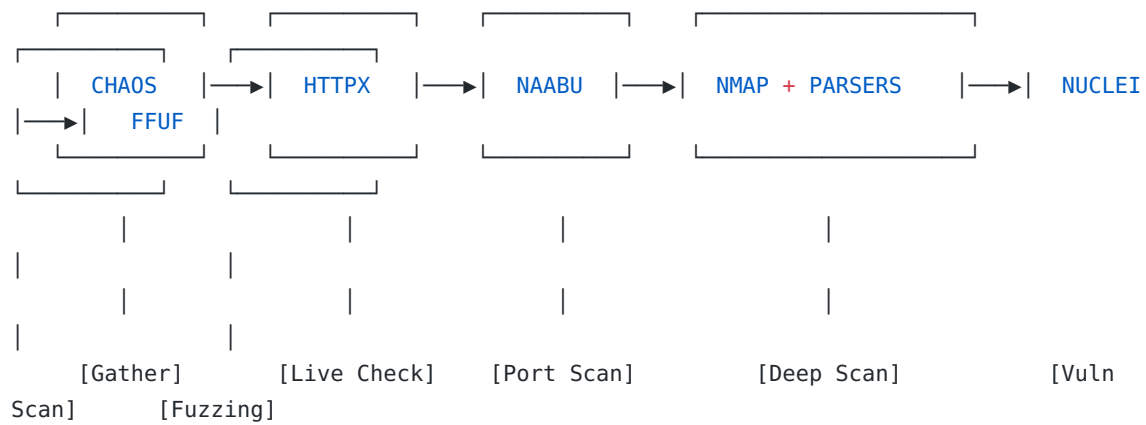
Introduction	3
Phase 1: Subdomain Discovery with Chaos	5
Phase 2: Identifying Alive Hosts and Deduplication	6
Probing with HTTPX:	6
IP Deduplication:	7
Phase 3: Port Scanning with Naabu	8
Phase 4: Advanced Service Detection & Nmap Parsing	8
Analyzing the Results	9
Phase 5: Automated Vulnerability Scanning with Nuclei	10
Phase 6: Content Discovery and Fuzzing with FFUF	11
Pro-Tips for Fuzzing:	11
Conclusion	12
Disclaimer	12

Introduction

In **Bug Bounty** hunting, the line between landing a duplicate and uncovering a real critical issue usually comes down to how you perform reconnaissance. A lot of hunters stay on the surface. They scan the obvious assets, check the common ports and move on. But the valuable findings are often hidden in places most people ignore, like non standard ports, forgotten IP ranges and services that have been quietly running for years without attention.

In this guide, I'll walk you through a practical, multi stage approach to scanning and fuzzing that helps you expand your attack surface properly and spot opportunities others miss.

Recon Workflow



Phase 1: Subdomain Discovery

- └─ Tool: Chaos
 - └─ Certificate transparency logs
 - └─ DNS PTR records
 - └─ TLS scans
- └─ Broaden external attack surface

Phase 2: Alive Hosts and Deduplication

- └─ Probing with HTTPX
 - └─ Identify live assets
 - └─ Collect IP addresses
- └─ IP Deduplication
- └─ CDN/WAF Filtering
 - └─ Avoid Cloudflare/Akamai/Fastly
 - └─ Verify origin via page titles

Phase 3: Port Scanning

- └─ Tool: Naabu
 - └─ Top 100 ports
 - └─ Verify open status
 - └─ Create baseline for scanning

Phase 4: Service Detection and Parsing

- └─ Advanced Nmap Scan
 - └─ Software identification
 - └─ Version fingerprinting
 - └─ NSE vulnerability scripts

- └─ Nmap Parsing
 - └─ Convert XML to HTML
 - └─ Host exposure summary
 - └─ Identify low hanging fruit

Phase 5: Automated Vulnerability Scanning

- └─ Tool: Nuclei
 - └─ Known CVE templates
 - └─ Default credentials
 - └─ Misconfigurations
 - └─ Exposed tokens

Phase 6: Content Discovery and Fuzzing

- └─ Tool: FFUF
 - └─ Directory brute forcing
 - └─ Hidden backup files
 - └─ Sensitive paths (/admin, .env)
- └─ Fuzzing Pro-Tips
 - └─ Analyze response size/word count
 - └─ Include non-standard ports
 - └─ Investigate 403 Forbidden

Phase 1: Subdomain Discovery with Chaos

The first step in any reconnaissance process is discovering as many subdomains as possible. Instead of depending only on active brute-forcing or managing multiple API keys from different providers, we start with **Chaos** a tool that collects subdomains from certificate transparency logs, DNS PTR records and TLS scans, then stores them in a continuously updated database.

This lets you gather a large set of known assets before you even send a single probe to the target.

```
chaos -d redbull.com -o redbull.txt
```

```

www.vhrdbsb1ai01.sap.redbull.com
www.videri.api.redbull.com
www.wcmupd.redbull.com
www.wcmupd-q.redbull.com
www.web01078.microsites01.redbull.com
www.web02056.microsites02.redbull.com
www.web03014.microsites03.redbull.com
www.web03045.microsites03.redbull.com
www.web09.microsites.redbull.com
www.whpics.redbull.com
www.winupdate.redbull.com
www.winupdate-q.redbull.com
www.worldwrite.redbull.com
www.worldwrite-staging.redbull.com
www.xc.redbull.com
www.zacptbrvr01rb.redbull.com
www2.redbull.com
x-cockpit.redbull.com
xc.redbull.com
xfighters.redbull.com
xivado.redbull.com
yallanasser.redbull.com
za.redbull.com
zacptbrvr01rb.redbull.com
zoe-staging-new.redbull.com
zoom-sdk-embed.redbull.com

[coffinxp@Lostsec]--[~/NewMethodology]
>> cat redbull.txt | wc -l
2803

```

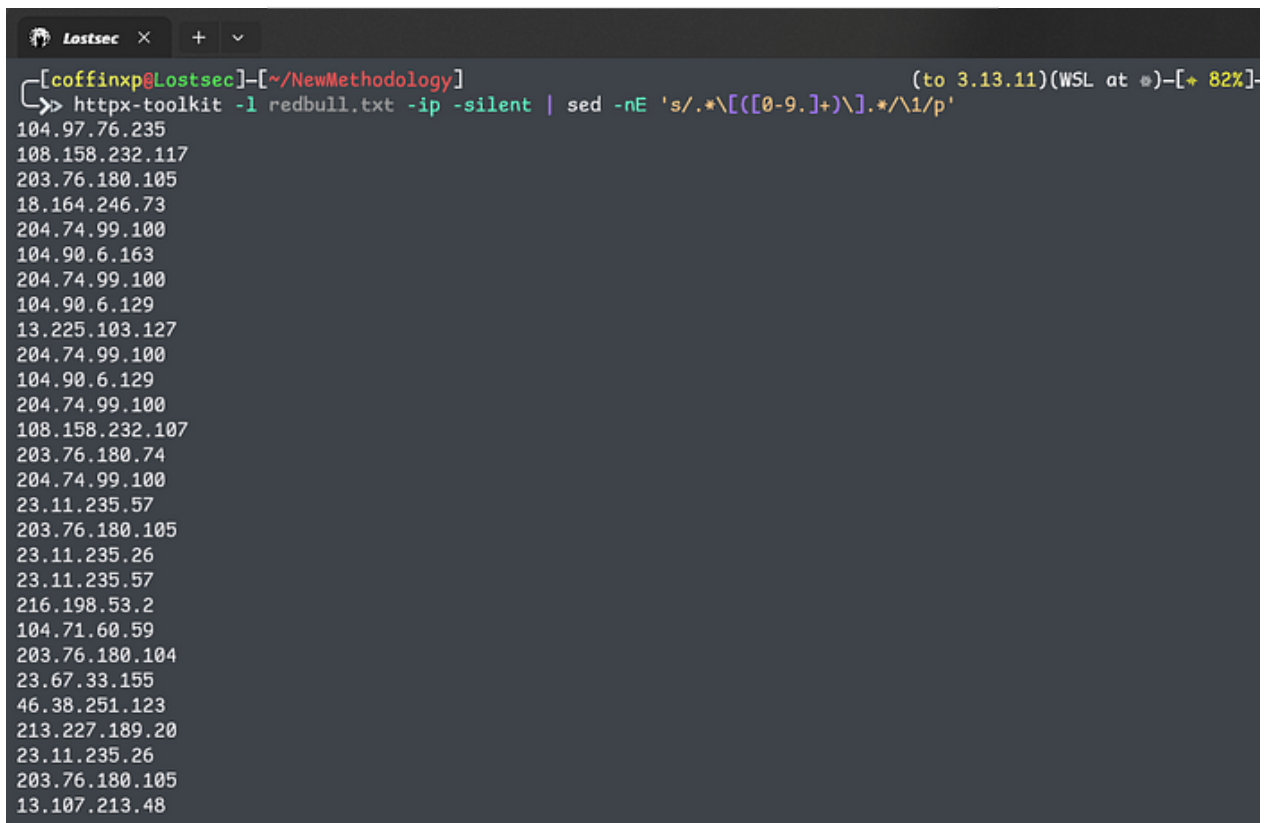
After running Chaos, we end up with a raw list of roughly 3,000 subdomains. That immediately gives us a much broader view of the target's external attack surface

Phase 2: Identifying Alive Hosts and Deduplication

A long list of subdomains means nothing if most of them don't resolve. Before moving forward, we have to separate the real, live assets from the dead entries. This step eliminates noise and lets you focus only on what actually respond and are worth testing.

Probing with HTTPX:

Next, we run the list through httpx to check which subdomains are actually alive. At the same time, we collect their corresponding IP addresses.



```

[coffinxp@Lostsec] - [~/NewMethodology] (to 3.13.11)(WSL at *) - [82%]-
>> httpx-toolkit -l redbull.txt -ip -silent | sed -nE 's/.*\([([0-9.]+)\).*\/\1/p'
104.97.76.235
108.158.232.117
203.76.180.105
18.164.246.73
204.74.99.100
104.90.6.163
204.74.99.100
104.90.6.129
13.225.103.127
204.74.99.100
104.90.6.129
204.74.99.100
108.158.232.107
203.76.180.74
204.74.99.100
23.11.235.57
203.76.180.105
23.11.235.26
23.11.235.57
216.198.53.2
104.71.60.59
203.76.180.104
23.67.33.155
46.38.251.123
213.227.189.20
23.11.235.26
203.76.180.105
13.107.213.48

```

IP Deduplication:

Many subdomains resolve to the same backend server or load balancer. To avoid wasting time, we sort the IPs and remove duplicates so we only scan unique hosts.

```
httpx-toolkit -l redbull.txt -ip -silent | sed -nE 's/.*\([([0-9.]+)\).*\/\1/p' | sort -u
>ip.txt
```

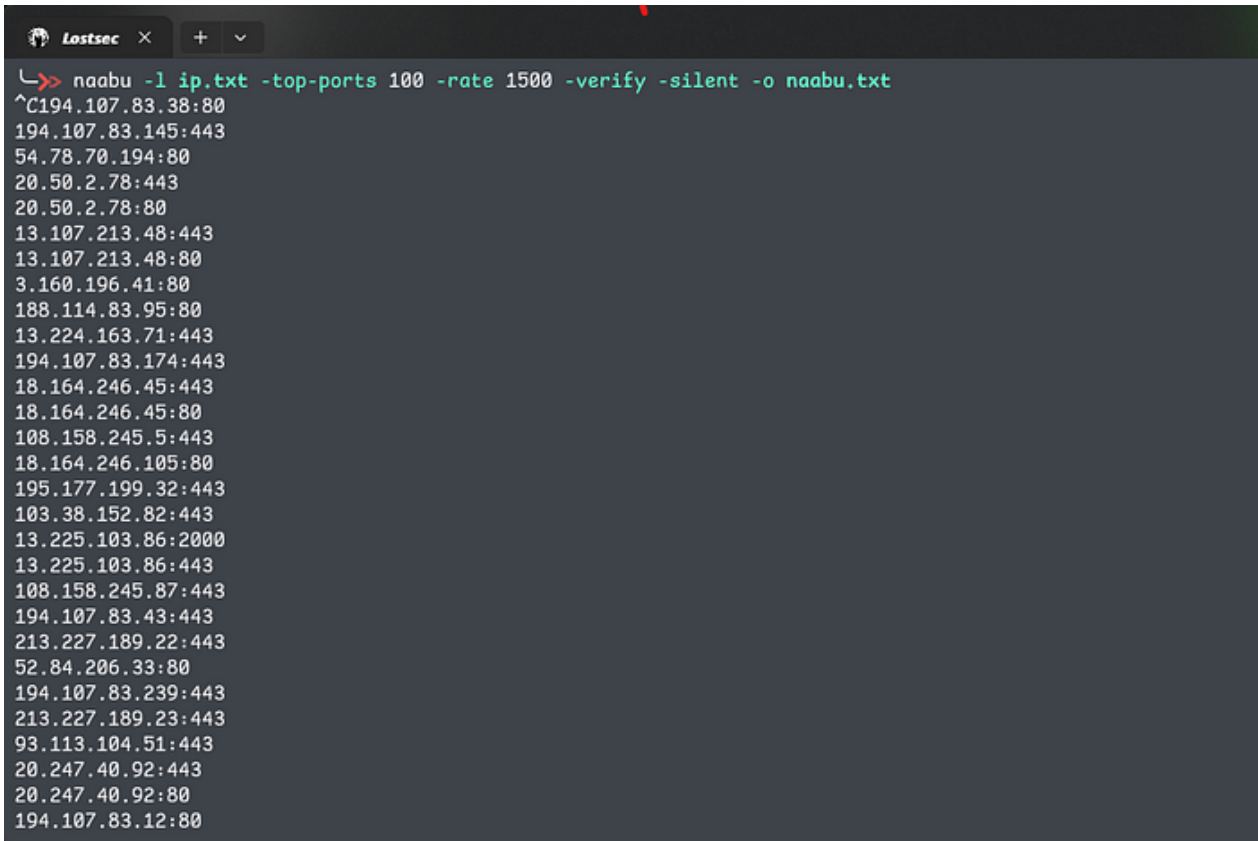
After cleaning the data, we're left with 379 unique IPs, a far more manageable and efficient list to work with.

❏Note: Before you scan or fuzz anything, check if the IP belongs to Cloudflare, Akamai or Fastly. Most of your collected IPs will. Scanning them is pointless you're hitting their edge servers, not the real origin. You'll also get your IP banned fast. so always use httpx with the -title flag to quickly see what the IP resolves to. If the title matches the actual website, it's likely the origin IP. If it shows generic CDN/WAF pages, it's not worth targeting.

Phase 3: Port Scanning with Naabu

Scanning every port across thousands of subdomains would be slow and noisy. Instead, running Naabu against the unique IPs lets you identify verified open ports quickly and efficiently.

```
naabu -l ip.txt -top-ports 100 -rate 1500 -verify -silent -o naabu.txt
```



```
lostsec x + v
L>> naabu -l ip.txt -top-ports 100 -rate 1500 -verify -silent -o naabu.txt
^C194.107.83.38:80
194.107.83.145:443
54.78.70.194:80
20.50.2.78:443
20.50.2.78:80
13.107.213.48:443
13.107.213.48:80
3.160.196.41:80
188.114.83.95:80
13.224.163.71:443
194.107.83.174:443
18.164.246.45:443
18.164.246.45:80
108.158.245.5:443
18.164.246.105:80
195.177.199.32:443
103.38.152.82:443
13.225.103.86:2000
13.225.103.86:443
108.158.245.87:443
194.107.83.43:443
213.227.189.22:443
52.84.206.33:80
194.107.83.239:443
213.227.189.23:443
93.113.104.51:443
20.247.40.92:443
20.247.40.92:80
194.107.83.12:80
```

this gives us a clean, verified list of open ports, so we are not wasting effort on filtered or closed services. Save this output, it becomes the baseline for deeper scanning.

Phase 4: Advanced Service Detection & Nmap Parsing

Now that we have a clean list of IPs and open ports, the next step is to understand what is actually running behind them.

Using a custom automation script, we pass the Naabu results into a more in-depth Nmap scan that performs service detection, version fingerprinting, and NSE vulnerability script audits:

[scripts/naabutonmap.py at main · coffinxp/scripts](https://github.com/coffinxp/scripts/blob/main/naabutonmap.py) (<https://github.com/coffinxp/scripts/blob/main/naabutonmap.py>) Contribute to coffinxp/scripts development by creating an account on GitHub.


```
python naabutonmap.py -i naabu.txt
```

```
[coffinxp@Lostsec]--[~/NewMethodology] (to 3.13.11)(WSL at *)-[67%]
>> python naabutonmap.py -i naabu.txt
Starting Nmap 7.98 ( https://nmap.org ) at 2026-01-21 15:37 +0300
Starting Nmap 7.98 ( https://nmap.org ) at 2026-01-21 15:37 +0300
Starting Nmap 7.98 ( https://nmap.org ) at 2026-01-21 15:37 +0300
Starting Nmap 7.98 ( https://nmap.org ) at 2026-01-21 15:37 +0300
Nmap scan report for server-13-225-103-35.de151.r.cloudfront.net (13.225.103.35)
Host is up, received user-set (0.0091s latency).

PORT      STATE SERVICE      REASON          VERSION
80/tcp    open  http         syn-ack ttl 249 Amazon CloudFront httpd
|_http-title: ERROR: The request could not be satisfied
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-server-header: CloudFront
|_http-csrf: Couldn't find any CSRF vulnerabilities.
|_http-dombased-xss: Couldn't find any DOM based XSS.
443/tcp   open  ssl/https    syn-ack ttl 249 CloudFront
|_http-server-header: CloudFront
|_http-csrf: Couldn't find any CSRF vulnerabilities.
|_http-title: ERROR: The request could not be satisfied
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-dombased-xss: Couldn't find any DOM based XSS.

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 103.55 seconds
Starting Nmap 7.98 ( https://nmap.org ) at 2026-01-21 15:39 +0300
```

- **Service Detection:** Identifying what software is running (e.g., Apache, Nginx, Redis).
- **Version Detection:** Finding specific software versions to look for known exploits.
- **Vulnerability Checks:** Running script-level audits.

Analyzing the Results

The scan produces several XML files, which are not exactly easy to review in raw form. To make the data readable, we run it through an Nmap parser. This converts the detailed XML output into a clean, structured HTML report that is much easier to analyze.

```
# Parse XML output into a readable HTML report
./nmap-parse-output /nmap-out/20260121_153728/nmap_out.xml html > scan.html
```

Nmap Scan Report - Scanned at Wed Jan 21 15:37:28 2026

Scan Summary | server-13-225-103-35.del51.r.cloudfront.net (13.225.103.35) | server-52-84-206-103.del54.r.cloudfront.net (52.84.206.103) | km36016.keymachine.de (87.118.84.176) | 162.159.140.98

Scan Summary

Nmap 7.96 was initiated at Wed Jan 21 15:37:28 2026 with these arguments:
 /usr/bin/nmap -u privileged-ras@vnc --version all --script vuln, default --open --reason --ps :ip:8080,8443,443,80 --oX nmap-out/20260121_153728/nmap_out_162.159.140.98.xml
 162.159.140.98

Verbosity: 0; Debug level 0
 Nmap done at Wed Jan 21 15:39:30 2026; 1 IP address (1 host up) scanned in 122.71 seconds

13.225.103.35 / server-13-225-103-35.del51.r.cloudfront.net

Address

- 13.225.103.35 (IPv4)

Hostnames

- server-13-225-103-35.del51.r.cloudfront.net (PTR)

Ports

Port	State (toggle closed [0] filtered [0])	Service	Reason	Product	Version	Extra info
80	open	http	synack	Amazon CloudFront httpd		
http-title	ERROR: The request could not be satisfied					
http-stored-xss	Couldn't find any stored XSS vulnerabilities.					
http-server-header	CloudFront					
http-csrf	Couldn't find any CSRF vulnerabilities.					
http-dombased-xss	Couldn't find any DOM based XSS.					
443	open	https	synack	CloudFront		
http-server-header	CloudFront					

Go to top
Toggle Closed Ports

```
+-----+
| Feature      | Benefit                                         |
+-----+
| Host Summary | Quickly see which IPs have the most exposure. |
| Version Info | Identify "low hanging fruit" like outdated software. |
| Visual Layout | Easily spot high-value attack surfaces for manual testing. |
+-----+
```

GitHub - ernw/nmap-parse-output: Converts/manipulates/extracts data from a Nmap scan output. (<https://github.com/ernw/nmap-parse-output>) Converts/manipulates/extracts data from a Nmap scan output. - ernw/nmap-parse-output

Phase 5: Automated Vulnerability Scanning with Nuclei

With a confirmed list of live IPs and open ports, we move into targeted vulnerability hunting. Nuclei works with thousands of templates covering known CVEs, default credentials, misconfigurations and exposed tokens.

```
# Scan live IPs for known CVEs
cat ip.txt | nuclei -tags cve -bs 200

# Scan all discovered ports (Naabu output)
cat naabu.txt | nuclei -tags cve -bs 200
```

By feeding it the Naabu output directly, we make sure every discovered port gets tested, not just the usual web ports like 80 and 443. This is important because many applications, admin panels, APIs or sensitive files and directories often hosted on non-standard ports, which are easy to miss if you only scan typical web services.

Phase 6: Content Discovery and Fuzzing with FFUF

The final stage of reconnaissance is directory brute forcing or fuzzing, with ffuf.

Using a targeted wordlist, we look for hidden files, backup archives such as .bak or .zip and sensitive paths like /admin or exposed configuration files such as .env. This step often uncovers overlooked endpoints that can lead to serious findings.

```
ffuf -w ip.txt:SUB -w payloads/backup_files_only.txt:FILE -u https://SUB/FILE -mc 200
-rate 50 -fs 0 -c
ffuf -w naabu.txt:SUB -w payloads/backup_files_only.txt:FILE -u https://SUB/FILE -mc 200
-rate 50 -fs 0 -c
```

```
[coffinxp@Lostsec]~[~/NewMethodology] (to 3.13.11)(WSL at e)-[77%]-[21,15:56]
ffuf -w ip.txt:SUB -w /home/coffinxp/payloads/backup_files_only.txt:FILE -u https://SUB/FILE -mc 200 -rate 50 -fs
0 -c

v2.1.0-dev

:: Method      : GET
:: URL         : https://SUB/FILE
:: Wordlist    : SUB: /home/coffinxp/NewMethodology/ip.txt
:: Wordlist    : FILE: /home/coffinxp/payloads/backup_files_only.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200
:: Filter      : Response size: 0

[Status: 200, Size: 23, Words: 3, Lines: 1, Duration: 163ms]
* FILE: 0.1.0.sql
* SUB: 18.158.11.80
```

[payloads/backup_files_only.txt at main · coffinxp/payloads](https://github.com/coffinxp/payloads/blob/main/backup_files_only.txt) (https://github.com/coffinxp/payloads/blob/main/backup_files_only.txt) Contribute to coffinxp/payloads development by creating an account on GitHub.

Pro-Tips for Fuzzing:

- **Analyze the Responses:** Don't just look at status codes. Pay attention to Response Size and Word Count. Sometimes a 200 OK is just a custom error page that happens to match the size of a real page.
- **Include Non-Standard Ports:** Use the Naabu port list within ffuf. Vulnerabilities are frequently found on internal services exposed on unusual ports.

- Don't Ignore 403s: A 403 Forbidden response often means you've found something sensitive. These endpoints are prime candidates for 403 bypass techniques.

You can also watch this video where I showed the complete practice of this method:

I

Conclusion

This structured workflow turns random scanning into focused, high-value reconnaissance. By validating assets, mapping real services and then applying vulnerability and content discovery, you eliminate noise and gain a clear, prioritized attack surface. It saves time, scales well and puts you in a strong position for effective manual testing and real bug discoveries.

Disclaimer

The content provided in this article is for educational and informational purposes only. Always ensure you have proper authorization before conducting security assessments. Use this information responsibly.